

HTML/CSS fundamentals

COMPLETE LEARNING GUIDE

What You'll Learn

HTML and CSS are the substrate every frontend framework eventually compiles down to, and a surprising share of "React bugs" or "layout bugs" learners hit later are really unresolved gaps in these fundamentals — a misunderstood box model, a specificity fight, or not knowing when to reach for flexbox versus grid. This guide covers semantic HTML as a structural and accessibility tool, the box model and why box-sizing changes how every width calculation behaves, how the cascade and specificity actually resolve competing rules, and the two layout systems (flexbox and grid) that replaced float-based and table-based layouts. It closes with positioning/stacking context and CSS custom properties for maintainable theming. Treat this as the foundation the rest of the frontend track assumes is solid.

1. Semantic HTML and Document Structure

Semantic elements (header, nav, main, article, section, aside, footer) describe what content *is*, not just how it looks. A screen reader, search engine crawler, or browser's built-in reader mode all rely on this structure to navigate a page meaningfully — a page built entirely from generic divs gives them nothing to hook into. Semantic tags also come with free behavior: a button is keyboard-focusable and triggers on both click and Enter/Space without any JavaScript; a div styled to look like a button gives you none of that for free. The rule of thumb: reach for the most specific element that matches the content's meaning, and only fall back to div/span when no semantic element fits.

- Use `<button>` for anything clickable that performs an action, and `<a href>` only for actual navigation — this alone fixes most keyboard-accessibility bugs.
- `<main>` should appear exactly once per page and wrap the primary content, letting assistive tech users skip repeated nav/header content.
- Heading levels (h1-h6) should nest in order and describe document outline, not be chosen for their default font size.

HTML

```
<body>
  <header>
    <nav aria-label="Main">
      <a href="/">Home</a>
      <a href="/courses">Courses</a>
    </nav>
  </header>
  <main>
    <article>
      <h1>Understanding the Box Model</h1>
      <p>Every element on the page is a rectangular box...</p>
    </article>
  </main>
  <footer>
    <p>&copy; 2026 Learn Being Forward</p>
  </footer>
</body>
```

2. The Box Model and Layout Fundamentals

Every rendered element is a box made of four layers, from the inside out: content, padding, border, and margin. By default (content-box), width/height apply only to the content area — padding and border are added on top, so an element styled width: 200px with 20px of padding and a 2px border actually occupies 244px. This trips up nearly every beginner's layout math. Setting box-sizing: border-box flips the model so width/height include padding and border, meaning a 200px-wide box stays 200px regardless of padding — which is why almost every production CSS reset applies border-box globally on day one.

- Vertical margins between block siblings collapse to the larger of the two, not their sum — a common source of 'missing' spacing bugs.
- display: inline elements ignore top/bottom margin and explicit width/height entirely; use inline-block or flex if you need those.
- The universal border-box reset (* { box-sizing: border-box; }) should be one of the first rules in any real stylesheet.

CSS

```
*, *::before, *::after {
  box-sizing: border-box;
}

.card {
  width: 300px;
  padding: 24px;
  border: 1px solid #ddd;
  /* with border-box, the card is still exactly 300px wide */
}
```

3. CSS Selectors, Specificity, and the Cascade

When multiple rules target the same element, the browser resolves the conflict using specificity, source order, and !important — in that priority order. Specificity is calculated as a tuple of (inline styles, IDs, classes/attributes/pseudo-classes, elements), compared left to right. An ID selector (#nav) always beats any number of class selectors; a class always beats an element selector. When specificity ties, the rule declared later in the stylesheet (or later in document order for equally-loaded stylesheets) wins. Understanding this is what lets you predict which rule applies without trial-and-error, and it's the direct fix for the classic 'my CSS isn't applying' complaint.

- Inline style="" attributes have the highest specificity short of !important and should be avoided in maintainable codebases for that reason.
- Prefer classes over IDs for styling — IDs are hard to override later and encourage specificity wars.
- !important should be a last resort; reaching for it repeatedly is a sign the selector architecture needs rethinking, not patching.

4. Flexbox for One-Dimensional Layout

Flexbox arranges items along a single axis (row or column) and excels at distributing space, aligning items, and handling content that can wrap or resize dynamically — navbars, button groups, card rows, centering a single element. Setting display: flex on a container turns its direct children into flex items governed by main-axis (justify-content) and cross-axis (align-items) alignment properties. flex-grow, flex-shrink, and flex-basis on individual items control how they expand, contract, and what size they start from when space is distributed — this trio is what actually determines 'who gets the extra space' in a flex row, and it's the part most learners skip past without understanding.

- justify-content controls alignment along the main axis; align-items controls the cross axis — mixing these up is the most common flexbox debugging mistake.
- flex: 1 is shorthand for flex-grow: 1; flex-shrink: 1; flex-basis: 0% — it makes an item grow to fill available space equally with siblings.
- gap works inside flex containers just like grid, and is now preferred over margin hacks for spacing between flex items.

CSS

```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 12px 24px;
}

.navbar .spacer {
  flex: 1; /* pushes following items to the right edge */
}
```

5. CSS Grid for Two-Dimensional Layout

Grid handles layout where both rows and columns matter simultaneously — page-level layouts, dashboards, image galleries, and any design where items need to align across both axes. `display: grid` combined with `grid-template-columns/rows` defines the track sizes, and the `fr` unit distributes remaining space proportionally. Unlike flexbox, grid lets you place items explicitly by line number or named area, and `repeat()/minmax()` let you build responsive grids without a single media query in many cases. The general heuristic: reach for flexbox when laying out a single row or column of items, and grid when the layout is genuinely a two-dimensional structure.

CSS

```
.dashboard {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 20px;
}

.page-layout {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar content"
    "footer footer";
  grid-template-columns: 220px 1fr;
}

.page-layout > header { grid-area: header; }
.page-layout > aside { grid-area: sidebar; }
```

6. Positioning and Stacking Context

`position: static` (default) leaves elements in normal document flow. `relative` positions an element offset from where it would normally sit while still reserving that original space. `absolute` removes the element from flow entirely and positions it relative to the nearest positioned ancestor (any ancestor with position other than `static`) — if none exists, it falls back to the document itself, which is the source of most 'why is my absolutely-positioned element in the wrong place' bugs. `fixed` positions relative to the viewport and stays put during scroll; `sticky` toggles between `relative` and `fixed` behavior based on scroll position. `z-index` only has an effect on positioned elements and only compares within the same stacking context.

- An absolutely positioned element needs a positioned ancestor (usually `position: relative` on the parent) to be contained — otherwise it positions against the whole page.
- Creating a new stacking context (via `transform`, `opacity < 1`, or `position + z-index`) can trap a child's `z-index` so it never appears above elements outside that context, regardless of how high the `z-index` value is.

7. CSS Custom Properties and Maintainable Styling

CSS custom properties (`--variable-name`) let you define values once and reference them throughout a stylesheet with `var()`, and — unlike preprocessor variables (Sass) — they're live in the browser, meaning they can be changed at runtime with JavaScript or overridden per-component/per-theme via the cascade. This is the mechanism behind most modern dark-mode implementations: define color tokens on `:root`, then override the same variable names inside a `[data-theme='dark']` selector, and every component that references `var(--bg-color)` updates automatically without touching component CSS.

CSS

```
:root {
  --color-primary: #2563eb;
  --color-bg: #ffffff;
  --spacing-md: 16px;
}

[data-theme="dark"] {
  --color-bg: #0f172a;
}

.card {
  background: var(--color-bg);
  padding: var(--spacing-md);
  border-color: var(--color-primary);
}
```

Common Pitfalls

- Not setting `box-sizing: border-box` globally, then fighting width calculations that don't account for padding and border.
- Using `div`s and `span`s styled to look like buttons/links instead of real `<button>`/`<a>` elements, silently breaking keyboard and screen-reader access.
- Reaching for absolute positioning to fix a layout problem that flexbox or grid would solve more robustly and responsively.
- Not giving an absolutely positioned element's parent `position: relative`, causing it to position against the entire document instead.
- Escalating specificity fights with more and more specific selectors (or `!important`) instead of restructuring the CSS architecture.
- Forgetting that margins collapse vertically between block siblings, then adding redundant padding/margin to 'fix' spacing that was never actually missing.
- Confusing flexbox's `justify-content` (main axis) with `align-items` (cross axis), especially after switching `flex-direction`.

Key Takeaways

- Semantic HTML gives you accessibility and SEO behavior for free — generic divs give you none of it.
- border-box should be a day-one global reset; content-box's default math is almost never what you want.
- Specificity resolves as (inline, IDs, classes, elements) compared left to right, with source order as the final tiebreaker.
- Flexbox is for one axis; grid is for two — most 'flexbox vs grid' confusion disappears once that distinction is internalized.
- position: absolute always positions against the nearest positioned ancestor, or the whole document if there isn't one.
- CSS custom properties are live and cascade-aware, making them the standard mechanism for theming today.

Learn More

MDN — CSS Box Model

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_box_model/Introduction_to_the_CSS_box_model

MDN — Basic Concepts of Flexbox

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox

CSS-Tricks — A Complete Guide to Grid

<https://css-tricks.com/snippets/css/complete-guide-grid/>